

Tap-on-Phone

Guia de integração para autenticação de sistemas de extensão do Tap-on-Phone

Versão 1.1

DEZEMBRO 2024

HISTÓRICO DE MUDANÇAS

DATA	AUTORES	VERSÃO	MUDANÇAS
27/09/2023	GABRIEL A. DUARTE	1.0	Criação da documentação
02/12/2024	GABRIEL A. DUARTE	1.1	Atualizações gerais

INTRODUÇÃO

1. Objetivo da documentação

Esta documentação tem por objetivo oferecer uma compreensão abrangente e clara da autenticação para aplicações que implementam soluções de extensão do **Tap-on-Phone**, aplicação desenvolvida para permitir pagamentos por aproximação, de cartões de crédito e débito diretamente nos celulares Android™, através de comunicação via NFC. Este documento visa auxiliar desenvolvedores, integradores e demais interessados na implementação bem-sucedida da integração com o sistema de autenticação para essa solução.



Objetivando eliminar obstáculos na implementação, na sequência serão encontrados tópicos que apresentam uma visão geral do sistema de autenticação, descrevendo seus componentes, funcionalidades e fluxos de interação. Também estão presentes os requisitos funcionais e não funcionais que permitem a criação de aplicações que não só sejam capazes de se autenticar para utilizar a solução de pagamento, mas que o façam com garantias de estabilidade, desempenho e principalmente segurança.

No contexto técnico, são descritos protocolos, padrões e exemplos que servirão de norte aos desenvolvedores, agregando celeridade à implementação das aplicações. Para que não sejam ignorados atributos relevantes, como a gestão das credenciais, métodos de acompanhamento e tráfego seguro de dados, indicações de boas práticas também fazem parte dessa documentação.

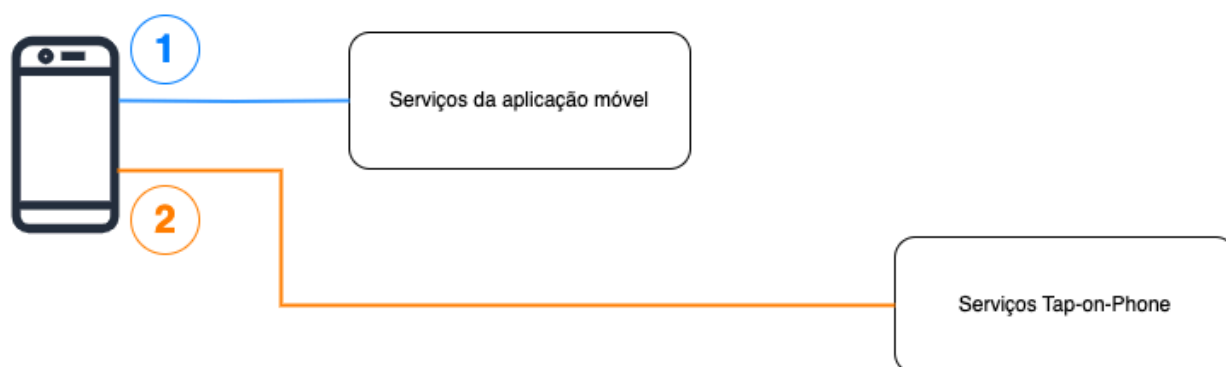
Por fim, para facilitar a compreensão da documentação, bem como auxiliar na solução de problemas futuros, ainda se encontram disponíveis algumas perguntas frequentes.

2. Visão geral da autenticação para sistemas de extensão do Tap-on-Phone

A autenticação para sistemas de extensão do **Tap-on-Phone** leva em consideração a autenticação prévia destas aplicações em seus próprios serviços, eliminando a necessidade de uma segunda autenticação na infraestrutura **Getnet / PagoNxt**. Em outras palavras, a autenticação migra de um processo que ocorre entre aplicação e os serviços **Tap-on-Phone** e passa a ser feita por uma integração entre servidores (Figura 2.1). Tudo isso se baseia na confiança que a infraestrutura remota da aplicação possui na solução que está embarcada no dispositivo dos usuários, sendo essa confiança previamente garantida por outras formas próprias de autenticação (tokens, certificados de segurança, assinaturas criptográficas).

Essa abordagem elimina a necessidade de múltiplos processos de autenticação na aplicação móvel, tornando a transação mais ágil e fluida. Sob o prisma dos pagamentos por aproximação em dispositivos Android™, esse padrão de autenticação é uma solução que se baseia na autenticação contínua da aplicação, mantendo a segurança do processo transacional e proporcionando uma experiência de usuário mais eficiente. Ao reconhecer e utilizar a autenticação da aplicação já existente, propicia-se uma jornada de pagamento mais direta e confiável para o usuário final.

Autenticação tradicional



Autenticação silenciosa da extensão do Tap-on-Phone



Figura 2.1 – Autenticação da extensão do Tap-on-Phone

Na Figura 2.1, enquanto a **Autenticação tradicional** exige dois passos distintos, sendo o passo 1 a autenticação da aplicação nos seus próprios serviços, e o passo 2, a autenticação da aplicação nos serviços do **Tap-on-Phone**, a **Autenticação da extensão do Tap-on-Phone** realiza o passo 2 internamente, no *backend* da aplicação, enquanto a aplicação ainda aguarda a resposta do passo 1. Dessa forma, do ponto de vista da aplicação, apenas um processo de autenticação acontece.

3. Público-alvo

Esta documentação é de interesse para os seguintes papéis relacionados às aplicações móveis:

- **Desenvolvedores:** responsáveis pela implementação da solução de extensão do **Tap-on-phone** para aplicativos móveis, que necessitam de informações detalhadas sobre a integração, configuração e melhores práticas.
- **Arquitetos:** responsáveis pela elaboração da arquitetura da solução, que irão planejar, projetar e implementar a autenticação em conformidade com os requisitos de segurança e desempenho.
- **Engenheiros de Segurança:** profissionais encarregados de garantir a segurança da aplicação e dos dados do usuário, que necessitam de insights sobre as melhores práticas de segurança para a solução de pagamento.

- **Gestores de Produto:** gestores responsáveis pela estratégia e *roadmap* dos produtos relacionados a pagamentos móveis, que buscam compreender os benefícios e funcionalidades da autenticação para aprimorar a experiência do usuário.
- **Profissionais de infraestrutura e operações:** envolvidos na implementação e manutenção da infraestrutura da aplicação, incluindo implementação, configuração e monitoramento das aplicações.
- **Equipe de suporte técnico:** profissionais de suporte que necessitam de conhecimento técnico para auxiliar na resolução de problemas e dúvidas que surjam durante a utilização da aplicação.
- **Consultores de segurança:** consultores externos que realizam auditorias de segurança ou fornecem recomendações para garantir a integridade e a segurança da aplicação.

REQUISITOS FUNCIONAIS

1. Autenticação da aplicação

A autenticação da aplicação é um requisito fundamental para garantir que a aplicação móvel seja devidamente identificada e autorizada para realizar transações. Esta funcionalidade é crucial para a segurança e integridade do sistema de pagamentos móveis, pois assegura que apenas aplicativos autorizados possam realizar operações financeiras, evitando exposição indevida de dados, prejuízos financeiros e danos à imagem das organizações. Na sequência estão alguns dos principais aspectos da autenticação das aplicações:

1.1. Identificação da aplicação:

O sistema deve ser capaz de identificar de forma única e precisa a aplicação móvel no momento da autenticação. Isso geralmente envolve o uso de identificadores únicos, chaves criptográficas (certificados de segurança), tokens de autenticação específicos da aplicação, ou combinações dessas alternativas.

De forma prática, os serviços da aplicação móvel devem implementar uma solução para identificar unitariamente as instalações dessa aplicação nos dispositivos dos usuários, garantindo que somente versões válidas e autênticas sejam liberadas como extensões válidas do **Tap-on-phone**.

1.2. Autorização da aplicação:

Além da identificação da aplicação, o sistema deve garantir que os aplicativos acessem somente os recursos para quais foram previamente elaborados, projetados e aplicados. Isso é alcançado por meio de validação de credenciais específicas da aplicação, e garante que outros recursos restritos não sejam livremente expostos. Exemplo: em um cenário hipotético, duas aplicações distintas, uma voltada a clientes e outra a gerência, acessam os mesmos recursos de um serviço bancário. É evidente, porém, que a aplicação gerencial possui acessos a alguns recursos, como a liberação de crédito, que não devem estar disponíveis na aplicação para os clientes. Isso não significa apenas ocultar as opções no *frontend* da aplicação móvel, mas resguardar no *backend* que esses recursos não estão acessíveis quando as credenciais da aplicação dos clientes são utilizadas.

Dessa forma, os serviços da aplicação móvel devem verificar se a aplicação em questão, e até mesmo o usuário, se for o caso, possuem acesso ao recurso de **Tap-on-phone**, antes de liberar o acesso aos respectivos serviços.

2. Autenticação nos serviços de GetTap:

A autenticação feita pelo *backend* da aplicação não exige a necessidade de autenticação na infraestrutura **Getnet / PagoNxt**, apenas transferindo essa responsabilidade para os serviços que sustentam a aplicação móvel. Isso propicia uma série de facilidades, como a já citada redução de uma autenticação feita diretamente pela aplicação, além de facilitar o gerenciamento de credenciais de segurança, agregar fluidez, segurança e observabilidade ao processo.

Assim sendo, além de autenticar a autorizar a aplicação presente nos dispositivos dos usuários, os serviços que sustentam a aplicação devem, internamente, realizar o processo de autenticação para **Tap-on-phone**. Após essa autenticação, as credenciais obtidas são então transmitidas a aplicação para que essa as utilize. São aspectos dessa etapa de autenticação:

2.1. Autenticação no single sign-on (SSO):

A funcionalidade de autenticação no Single Sign-On (SSO) permite que o *backend* da aplicação seja identificado nos serviços **Tap-on-phone**. Quando é realizada a autenticação, os serviços de SSO identificam esse *backend* como um usuário sistêmico e concede o token de acesso que deve ser utilizado para chamar todos os outros serviços integrados, incluindo as operações do **Tap-on-phone**. Isso simplifica significativamente o processo, facilitando uma gestão centralizada de acesso, ao mesmo tempo em que mantém os padrões de segurança necessários para proteger os dados e garantir um ambiente confiável para todas as operações.

Assim, o primeiro passo do fluxo de autenticação é, de fato, a chamada dos serviços de SSO para a obtenção do token de acesso.

2.2. Autenticação no serviço de GetTap:

Os serviços do **Tap-on-phone** funcionam em um ecossistema apartado, isolando seus recursos para garantir a segurança e inviolabilidade das suas operações. Assim sendo, uma série de verificações ocorrem para garantir que a aplicação que esteja tentando se autenticar possui realmente todos os atributos que a qualificam como válida para as operações nesse ambiente. Isso faz com que, após a autenticação no SSO, seja necessária uma segunda autenticação, agora exclusiva para **Tap-on-phone**.

Dessa forma, a última etapa do processo de autenticação ocorre quando o *backend*, previamente detentor do token obtido no SSO, chama a autenticação nos serviços do **Tap-on-phone** obtendo o segundo token necessário (*app-token*). Ambos devem ser enviados à aplicação para que essa possa, então, iniciar as operações solicitadas pelo usuário.

REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais, muitas vezes chamados de requisitos de qualidade, descrevem as características do sistema que não estão diretamente relacionadas às funcionalidades específicas que o sistema deve oferecer, mas sim às qualidades gerais que o sistema deve ter. Eles definem as condições sob as quais o sistema deve operar, em vez do que o sistema deve fazer. Esses requisitos influenciam a arquitetura do sistema, a usabilidade, a segurança, o desempenho e outros aspectos importantes. Os

principais requisitos não funcionais indicados para os serviços que irão utilizar a solução de **Tap-on-phone** são:

1. Segurança

Devem ser observados, criteriosamente, todos os requisitos de segurança de dados, proteção contra ameaças e acesso não autorizado, criptografia, controle de acesso e políticas de segurança. Isso se traduz em processos multidisciplinarmente projetados e validados para suprimir todas as possíveis violações que a aplicação possa sofrer, mitigando possíveis ataques. Os próprios processos de autenticação estão diretamente ligados a esse aspecto, além da utilização de comunicações via SSL, uma gestão confiável das credenciais de segurança, monitoramento de vulnerabilidade dos códigos e dependências, entre outras questões.

2. Desempenho

Não basta que os serviços sejam plenamente seguros e capazes de realizar as operações para as quais foram propostos. É necessário que os serviços sejam capazes de atender a uma demanda geralmente crescente de requisições, mantendo o tempo de resposta das chamadas em níveis adequados. Por exemplo, o sistema deve ser capaz de processar 1000 transações por segundo, mantendo o tempo de resposta inferior a 100 microssegundos.

3. Disponibilidade

Requisitos relacionados ao tempo em que o sistema está apto a atender as requisições, bem como a tolerância a falhas e capacidade de recuperação. De forma prática, é comum se estabelecer um valor mínimo para a disponibilidade, como definir que o sistema deve estar 99,9% do tempo disponível, ao longo do ano. Um sistema que não atende aos critérios de disponibilidade tende a incomodar os usuários, que muitas vezes deixam de utilizar os recursos e podem até migrar para concorrentes em busca de uma maior estabilidade.

CRITÉRIOS PARA UTILIZAÇÃO DOS SERVIÇOS DE TAP-ON-PHONE

Para garantir a utilização segura e adequada dos serviços disponibilizados, é imperativo que a aplicação atenda a critérios específicos de validação e registro. Recomenda-se que esses critérios sejam atendidos antes mesmo do início do desenvolvimento dos serviços de autenticação no *backend* da aplicação móvel, facilitando validações e testes.

1. Cadastro no Single Sign-On (SSO):

A aplicação deve estar devidamente cadastrada no sistema Single Sign-On (SSO) como um usuário sistêmico. Esse processo de cadastro, já mencionado anteriormente, proporciona uma identificação única e segura, permitindo a autenticação adequada na plataforma.

2. Cadastro nos serviços de Tap-on-phone:

A aplicação necessita estar registrada na plataforma **Tap-on-Phone**, além de estar associada à empresa responsável por ela. Em outras palavras, uma empresa pode possuir várias aplicações diferentes utilizando a solução de pagamento, desde que todas essas aplicações estejam cadastradas como pertencentes a essa organização. Este cadastro assegura que a aplicação seja reconhecida como uma entidade autorizada a realizar operações dentro do ecossistema de pagamentos para aquela empresa em questão.

3. Cliente ativo e apto a realizar transações:

A empresa que visa transacionar via **Tap-on-phone** deve ser um cliente ativo e devidamente habilitado para operações na plataforma de pagamentos. Esse critério é crucial para garantir que as transações possam ser efetuadas de forma legal, com questões contratuais válidas e sendo respeitadas, e em conformidade com as políticas estabelecidas.

ARQUITETURA

4. Componentes do sistema

Conforme já mencionado, a arquitetura do sistema de autenticação é composta por componentes internos e externos a infraestrutura **Getnet / PagoNxt**. A escolha da utilização de cada um desses componentes foi realizada com o objetivo principal de agregar segurança ao fluxo de pagamento, sem onerar, por conta disso, a usabilidade e o desempenho das aplicações. O componente foco dessa documentação são os serviços que sustentam a aplicação móvel, implementados e providos pelas organizações parceiras, e que receberão os recursos de autenticação para aplicações que utilizam o recurso de **Tap-on-Phone**.

Um ponto importante é que não há necessidade, e não é recomendável, que sejam construídas soluções exclusivas para a autenticação das aplicações. Os recursos de integração necessários para a autenticação devem ser acrescentados aos próprios serviços que já provêm as outras funcionalidades da aplicação, assim garantindo que todas as técnicas e metodologias de segurança prévias abranjam também essa nova funcionalidade. Essa recomendação vai ao encontro da necessidade de confiança entre infraestrutura e aplicação, descritas no **Tópico 2 - Visão geral da autenticação para sistemas de extensão do Tap-on-Phone**.

São componentes do sistema de autenticação:

- Componentes do parceiro:
 - **Aplicação do parceiro:** a aplicação móvel é a interface principal que os usuários utilizam para acessar e interagir com a plataforma de pagamentos por aproximação. Ela é instalada nos dispositivos dos usuários e serve como o ponto de início para as transações de pagamento.
 - **Serviços da aplicação móvel:** conjunto de serviços que sustentam as funcionalidades da aplicação móvel. Incluem, por exemplo, listagem de transações, consulta de saldo, configurações. A esse conjunto devem ser somados os recursos de autenticação para **Tap-on-Phone**.

- Serviços **Getnet / PagoNxt**:
 - o **Extensão do Tap-on-phone**: a extensão do **Tap-on-Phone** é responsável por realizar, de fato, o reconhecimento da aproximação do cartão e processamento da transação, sendo chamada pela aplicação móvel do parceiro quando o usuário solicitar que o pagamento seja iniciado.
 - o **Serviços de Single Sign-On (SSO)**: serviço que permite a autenticação singular de usuários e sistemas que acessam recursos da infraestrutura **Getnet / PagoNxt**.
 - o **Serviços de autenticação Tap-on-Phone**: são os recursos que farão o reconhecimento da aplicação como sendo válida para realizar transações via **Tap-on-Phone**.

5. Fluxo de autenticação

Uma vez compreendidas as motivações, os requisitos e os componentes da autenticação, é possível explorar, em detalhes, cada etapa do seu fluxo (Figura 5.1). Vale lembrar que essas etapas se resumem tecnicamente em requisições e respostas entre os componentes do sistema.

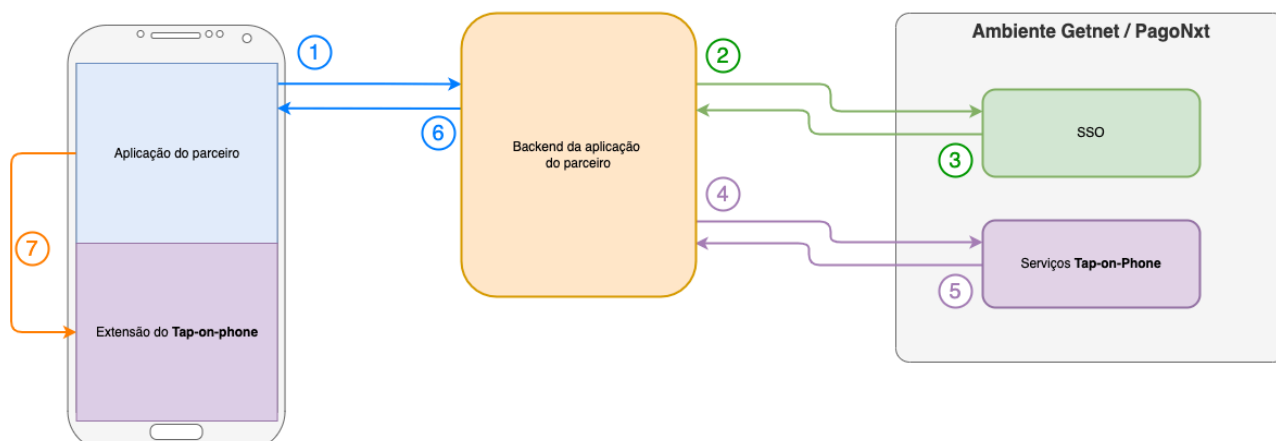


Figura 5.1 – Fluxo completo de autenticação

Conforme ilustrado na figura acima, o fluxo de autenticação é composto pelos seguintes passos:

1) Solicitação da autenticação iniciada pela aplicação

A aplicação móvel envia uma solicitação para iniciar o processo de autenticação ao serviço correspondente no seu *backend*. Essa requisição deve conter todos os dados necessários para que o serviço que sustenta a aplicação a reconheça como sendo autêntica e apta a receber as credenciais que serão obtidas nos próximos passos.

Importante: esse processo de autenticação é de responsabilidade do *backend* da aplicação, não havendo, ainda, quaisquer ligações ou validações feitas nos serviços **Getnet / PagoNxt**. A não garantia de autenticidade da aplicação, dada por uma autenticação feita sem os devidos critérios de segurança e controle, pode acarretar a entrega indevida das credenciais de **Tap-on-phone** para aplicações não autorizadas.

2) Solicitação da autenticação feita pelo serviço da aplicação para o SSO

O serviço da aplicação, após receber a solicitação de autenticação e efetuar as devidas validações, garantindo que a aplicação é realmente válida, envia uma requisição de autenticação ao Single Sign-On (SSO). Essa requisição deverá conter as credenciais de acesso armazenadas pelo *backend* da aplicação, que serão avaliadas pelo SSO para garantir que são válidas e estão ativas, correspondendo ao usuário sistêmico da aplicação.

Importante: as credenciais de acesso utilizadas para validação no SSO não devem, em nenhum momento, serem compartilhadas com a aplicação. A sua posse segura e restrita é uma das justificativas da utilização da autenticação para **Tap-on-phone** no *backend*.

3) Resposta do SSO para o serviço da aplicação

Uma vez comprovada a autenticidade das credenciais, mediante validação interna no SSO, este irá retornar o token de acesso para os recursos **Getnet / PagoNxt**. Este token deve ser utilizado no acesso a todos os recursos posteriores, inclusive devendo ser compartilhado com a aplicação.

4) Solicitação da autenticação para Tap-on-phone

Tendo recebido o token do SSO, o *backend* da aplicação se torna apto a realizar a autenticação exclusiva para **Tap-on-phone**. Para isso, será feita uma nova solicitação, com destino aos serviços de **Tap-on-phone**, por meio de uma requisição que deverá conter em seu cabeçalho, o token de acesso, e em seu corpo as informações de identificação da empresa (CNPJ) e da aplicação nome do pacote.

5) Resposta da autenticação de Tap-on-phone

Caso a empresa e a aplicação sejam reconhecidas como válidas, e estejam associadas no ambiente **Tap-on-phone**, um segundo token deverá ser retornado, liberando então o acesso aos recursos específicos para esse tipo de transação.

6) Resposta do serviço da aplicação

Se tudo ocorrer conforme o esperado e ambos os tokens forem fornecidos pelos serviços **Getnet / PagoNxt**, significando que a aplicação foi, de fato, reconhecida como válida a executar as operações de **Tap-on-Phone**, é necessário que o *backend* envie então esses dados para a aplicação embarcada no dispositivo do usuário. Nesse retorno devem estar contidos ambos os tokens, que deverão ser guardados de maneira segura na memória da aplicação.

7) Envio das credenciais de acesso da aplicação principal para a aplicação Getnet / PagoNxt

Por fim, considerando que a aplicação de extensão possua os tokens de acesso aos serviços, faz-se necessário que eles sejam repassados internamente para a aplicação **Tap-on-phone**, que realizará a transação. Isso ocorre quando o usuário solicita o início da operação na sua aplicação mobile.

DETALHES TÉCNICOS DA INTEGRAÇÃO

1. Pré-requisitos

Antes de iniciar a implementação do processo de autenticação, alguns pré-requisitos técnicos revelam-se importantes. Esses pré-requisitos, que se somam aos citados no tópico **CRITÉRIOS PARA UTILIZAÇÃO DOS SERVIÇOS DE TAP-ON-PHONE**, são:

1.1. Ambiente computacional disponível em rede

Disponibilidade de um ambiente computacional funcional e conectado a rede, onde os serviços serão executados, garantindo a comunicação eficaz entre os componentes do sistema. Esse ambiente, independentemente de estar hospedado localmente ou na nuvem, deve ser seguro, estável e resiliente para garantir a disponibilidade e confiabilidade dos serviços relacionados à autenticação.

1.2. Capacidade de acesso as APIs de autenticação

Além de estar plenamente acessíveis aos dispositivos que contém a aplicação mobile, os serviços hospedados no ambiente computacional devem possuir a capacidade de acessar e utilizar as APIs de autenticação necessárias para interagir com os serviços de autenticação, considerando que as requisições com destino ao SSO e ao recurso de autenticação **Tap-on-Phone** partir desse ponto. Isso se traduz em uma boa especificação sobre liberações de rede, permitindo que os protocolos, endereços e portas de comunicação demandados estejam liberados.

1.3. Gestão das credenciais de acesso

Devem ser previamente estabelecidas práticas e procedimentos para a gestão segura de credenciais de acesso usadas no processo de autenticação e comunicação segura, mitigando assim o risco de acessos não autorizados e garantindo a proteção dos dados sensíveis, bem como da inviolabilidade da operação como um todo.

1.4. Observabilidade e monitoramento

O ambiente, os serviços e todos os componentes relacionados, devem ser plenamente observáveis, possibilitando o monitoramento contínuo do desempenho, da segurança e de outros aspectos críticos para identificação precoce de problemas e otimização do sistema. Em outras palavras, políticas, processos e ferramentas de observabilidade, metrificação e monitoramento devem ser aplicados para gestão segura dos componentes da autenticação.

2. Protocolos e padrões de comunicação

Conhecer os protocolos e padrões de comunicação são fundamentais para a troca segura e eficiente de informações entre os componentes do sistema de autenticação. Eles estabelecem as regras e convenções

que permitem a comunicação e a transmissão de dados de maneira padronizada e confiável. Nesta implementação, alguns protocolos relevantes incluem:

2.1. HTTPS (Hypertext Transfer Protocol Secure):

Amplamente utilizado para comunicação segura na web, o HTTPS oferece uma camada adicional de segurança criptografando os dados trocados entre a aplicação móvel e os servidores, garantindo a integridade e a confidencialidade das informações. Atualmente sua aplicação é um padrão de segurança indispensável para recursos que trafegam dados na web.

2.2. REST (Representational State Transfer):

Padrão arquitetural que utiliza os métodos HTTP para comunicação assíncrona entre sistemas distribuídos. REST enfatiza a escalabilidade, confiabilidade e eficiência na transmissão de dados por meio de APIs, tornando-se uma escolha popular para integrações de serviços. Tanto o serviço de autenticação do SSO, quanto o serviço de autenticação para **Tap-on-Phone**, utilizam esse padrão em seu funcionamento.

2.3. OAuth (Open Authorization):

Protocolo de autorização que permite que a aplicação obtenha acesso a recursos em nome do usuário, com sua permissão. É crucial para a autorização segura da aplicação no processo de autenticação. Este protocolo é utilizado pelo SSO para autenticação do usuário sistêmico da aplicação.

2.4. JWT (JSON Web Tokens):

O padrão JWT é utilizado para geração dos tokens de acesso, sendo frequentemente utilizado nos processos de autenticação. Os JWTs são usados para transmitir informações de forma segura entre as partes, onde é possível garantir, através de criptografia, que as informações ali presentes são realmente fidedignas. Vale lembrar que o conteúdo dos tokens JWT não são criptografados de maneira a serem inacessíveis, ou seja, é possível para qualquer um que possua o token, ler o seu conteúdo, sendo garantida apenas a veracidade destes dados.

3. APIs de autenticação

Observação: É recomendável que, antes de iniciarem a implementação das integrações, os desenvolvedores confirmem a documentação OpenApi (Swagger) que acompanha este manual.

3.1. API SSO para obtenção do token

3.1.1. URLs de acesso:

Desenvolvimento: <https://apigee-gtw-hg.auttar.com.br/v1/tef-embarcado/oauth/token>

Homologação: <https://api-hti.auttar.com.br/v1/tef-embarcado/oauth/token>

Produção: <https://api.auttar.com.br/v1/tef-embarcado/oauth/token>

3.1.2. Autorização:

A autorização para esse recurso é feita utilizando **Basic Auth**, informando as credenciais do SSO codificadas em *Base64* no header **Authorization**. Exemplos:

Node JS:

```
const username = 'seu_usuario';
const password = 'sua_senha';

const credentials = `${username}:${password}`;
const encodedCredentials = Buffer.from(credentials).toString('base64');
const authHeader = `Basic ${encodedCredentials}`;
```

Java:

```
import java.util.Base64;

...

String username = "seu_usuario";
String password = "sua_senha";

String credentials = username + ":" + password;
String encodedCredentials = Base64.getEncoder().encodeToString(credentials.getBytes());

String authHeader = "Basic " + encodedCredentials;
```

Python:

```
import base64

username = 'seu_usuario'
password = 'sua_senha'

credentials = f'{username}:{password}'
encoded_credentials = base64.b64encode(credentials.encode('utf-8')).decode('utf-8')

auth_header = f'Basic {encoded_credentials}'
```

1.1.1. Requisição:

Havendo sido gerado o valor do *header Authorization*, é possível realizar a requisição. Exemplos:

Node JS:

```
import axios from 'axios';

...

const requestBody = new URLSearchParams();
requestBody.append('grant_type', 'client_credentials');

const response = await axios.post('<URL do SSO>', requestBody, {
  headers: {
    'Content-Type': 'application/x-www-form-urlencoded',
    'Authorization': authHeader
  }
});
```

Java:

```
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.StandardCharsets;

...

String requestBody = "grant_type=client_credentials";

URL url = new URL("<URL do SSO>");
HttpURLConnection connection = (HttpURLConnection) url.openConnection();
connection.setRequestMethod("POST");
connection.setRequestProperty("Content-Type", "application/x-www-form-urlencoded");
connection.setRequestProperty("Authorization", authHeader);
connection.setDoOutput(true);

try (OutputStream outputStream = connection.getOutputStream()) {
    outputStream.write(requestBody.getBytes(StandardCharsets.UTF_8));
}
```

Python:

```
import requests

...

headers = {
    'Content-Type': 'application/x-www-form-urlencoded',
    'Authorization': auth_header
}

response = requests.post(<URL do SSO>, data=request_data, headers=headers)
response.raise_for_status()
token_data = response.json()
```

1.1.1. Resposta:

Se a requisição for bem-sucedida, teremos em sua resposta o valor do token de acesso no seguinte formato:

```
{
  "access_token": "<TOKEN>",
  "expires_in": 3600,
  "token_type": "Bearer",
  "scope": "tap-on-phone"
}
```

Abaixo exemplos de como capturar e utilizar essa resposta, quando obtida pelos exemplos anteriores:

Node JS:

```
...

const token = response.data.access_token
```

Java:

```
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;

...

BufferedReader in = new BufferedReader(
    new InputStreamReader(connection.getInputStream())
);

StringBuilder response = new StringBuilder();
```

```
String inputLine;

while ((inputLine = in.readLine()) != null) {
    response.append(inputLine);
}
in.close();

String data = response.toString();

JsonObject jsonResponse = JsonParser.parseString(response.toString()).getAsJsonObject();

String accessToken = jsonResponse.get("access_token").getString();
```

Python:

```
token = token_data.get('access_token')
```

1.2. API Tap-on-Phone para obtenção do app-token

1.2.1. URLs de acesso:

Desenvolvimento: <https://apigee-gtw-hg.auttar.com.br/v1/tef-embarcado/top-auth-adp/app-token>

Homologação: <https://api-hti.auttar.com.br/v1/tef-embarcado/top-auth-adp/app-token>

Produção: <https://api.auttar.com.br/v1/tef-embarcado/top-auth-adp/app-token>

1.2.2. Autorização:

A autorização para esse recurso é feita informando o token obtido no SSO através do *header* **Authorization**.

1.1.2. Requisição:

Após o processo de autorização no SSO e a correta obtenção do token de acesso, é possível realizar a requisição específica para **Tap-on-Phone**, conforme exemplos:

Node JS:

```
import axios from 'axios';

...

const requestBody = {
  "document": "<CNPJ DA EMPRESA>",
  "package_name": "<PACOTE DE APLICAÇÃO>"
}

const response = await axios.post('<URL de AUTENTICAÇÃO GET TAP>', requestBody, {
  headers: {
    'Authorization': ssoToken
```



```
}  
});
```

Java:

```
...  
  
JsonObject requestBodyJson = new JsonObject();  
requestBodyJson.addProperty("document", "<CNPJ DA EMPRESA>");  
requestBodyJson.addProperty("package_name", "<PACOTE DE APLICAÇÃO>");  
  
String requestBody = requestBodyJson.toString();  
  
URL url = new URL(URL de AUTENTICAÇÃO GET TAP);  
HttpURLConnection connection = (HttpURLConnection) url.openConnection();  
connection.setRequestMethod("POST");  
connection.setRequestProperty("Content-Type", "application/json");  
connection.setRequestProperty("Authorization", authHeader);  
connection.setDoOutput(true);  
  
try (OutputStream outputStream = connection.getOutputStream()) {  
    outputStream.write(requestBody.getBytes(StandardCharsets.UTF_8));  
}
```

Python:

```
...  
  
request_data = {  
    "document": "<CNPJ DA EMPRESA>",  
    "package_name": "<PACOTE DE APLICAÇÃO>"  
}  
  
headers = {  
    'Content-Type': 'application/json',  
    'Authorization': auth_header  
}  
  
response = requests.post(URL de AUTENTICAÇÃO GET TAP, data=json.dumps(request_data), headers=headers)  
response.raise_for_status()  
token_data = response.json()
```

1.2.3. Resposta:

Se a requisição for bem-sucedida, receberemos o segundo token necessário na seguinte estrutura:

```
{  
    "httpStatusCode": 201,  
    "code": 0,  
    "message": "successful",  
    "details": {
```

```
    "token": "<TOKEN>"
  }
}
```

Abaixo exemplos de como capturar e utilizar essa resposta, quando obtida pelos exemplos anteriores:

Node JS:

```
...
const app_token = response.data.details.token
```

Java:

```
...
BufferedReader in = new BufferedReader(
    new InputStreamReader(connection.getInputStream())
);
StringBuilder response = new StringBuilder();
String inputLine;

while ((inputLine = in.readLine()) != null) {
    response.append(inputLine);
}
in.close();

String data = response.toString();

JsonObject jsonResponse = JsonParser.parseString(response.toString()).getAsJsonObject();
JsonObject details = jsonResponse.get("details");

String appToken = details.get("token").getString();
```

Python:

```
token_data = response.json()

details = token_data.get('details')
app_token = details.token;
```

Após a obtenção da resposta, ambos os valores do token SSO e token **Tap-on-Phone** devem ser transmitidos para a aplicação, finalizando assim o processo da autenticação.

Referências e solução de problemas

1. FAQ

1.1. Como funciona a autenticação para Tap-on-Phone?

Durante o processo principal de autenticação da aplicação, internamente o *backend* realiza a autenticação nos serviços de **Tap-on-Phone**, entregando as credenciais de acesso todas juntas para que a aplicação possa utilizar posteriormente, no acesso às suas funcionalidades.

1.2. Quais são os benefícios do processo diferenciado de autenticação para Tap-on-Phone?

O processo de autenticação para **Tap-on-Phone** agrega fluidez e segurança ao eliminar a necessidade de repetidas autenticações. Isso agiliza a operação e mantém um nível adequado de segurança, pois a autenticação é realizada em “segundo plano” (no *backend*).

1.3. Quais são os requisitos para implementar a autenticação para Tap-on-Phone?

Os pré-requisitos incluem um ambiente computacional seguro e estável, capacidade de acesso às APIs de autenticação, e ser um cliente ativo na base **Getnet / PagoNxt**, tendo a sua aplicação cadastrada nos serviços de SSO e **Tap-on-Phone**.

1.4. Quais são os protocolos de comunicação utilizados na autenticação Tap-on-Phone?

Os protocolos comuns incluem HTTPS, OAuth, JWT, e REST. Esses protocolos garantem a segurança e a eficiência da comunicação entre os componentes do sistema.

1.5. A autenticação feita no *backend* é uma abordagem segura?

Sim, a autenticação feita pelo *backend* é uma abordagem segura quando implementada corretamente, seguindo as práticas de segurança recomendadas, como criptografia, gestão segura de chaves e proteção adequada das credenciais.

1.6. Posso implementar esse tipo de autenticação na nuvem?

Sim, a implementação da autenticação pode ser feita tanto em ambientes locais (*on premise*) quanto em ambientes de nuvem, desde que os requisitos de disponibilidade em rede, segurança e observabilidade sejam atendidos.